

Лекция 2. DFS на неориентированных графах. BFS

1 Дерево обхода DFS

Определение 1.1. Деревом обхода DFS называют граф, состоящий из вершин, посещаемых в ходе обхода DFS, и следующих рёбер:

- Древесное ребро — ребро, по которому DFS переходит напрямую (переходы в белые вершины из серых);
- Обратное ребро — ребро, которое DFS просматривает, но не идёт по нему (переходы в серые вершины).

2 Функция t_{up}

Определение 2.1. Функцией $t_{up}(v)$ назовём функцию, определяемую следующим образом:

$$t_{up}(v) = \min \begin{cases} t_{in}(v), \\ \min_u t_{in}(u) \end{cases}$$

где u — предок v и при этом u достижима по обратному ребру из w — вершины поддерева v .

Псевдокод:

```

1 void FillTUP(Graph graph, Vertex from, Vertex ancestor) {
2     visited[from] = true;
3     t_in[from] = t_up[from] = timer++;
4     for (Vertex to : graph.GetNeighbours(from)) {
5         if (to == ancestor) {
6             continue;
7         }
8         if (visited[to]) {
9             t_up[from] = min(t_up[from], t_in[to]);
10        } else {
11            FillTUP(graph, to, from);
12            t_up[from] = min(t_up[from], t_up[to]);
13        }
14    }
15 }

```

3 Мосты и рёберная двусвязность

Определение 3.1. Две вершины в неориентированном графе называются рёберно двусвязными, если между ними есть два рёберно непересекающихся пути.

Теорема 3.1 (б/д). Отношение рёберной двусвязности является отношением эквивалентности.

Доказательство. Обозначим отношение рёберной двусвязности как $(\sim)$. Покажем, что $(\sim)$ является отношением эквивалентности.

Рефлексивность: Положим по определению $u \sim u$.

Симметричность: Покажем, что если $u \sim v$, то $v \sim u$. Действительно, пути в неориентированном графе не имеют направления, а значит если есть два рёберно непересекающихся пути из u в v , то те же самые пути (пройденные в обратном порядке) являются двумя рёберно непересекающимися путями из v в u .

Транзитивность: Пусть $u \sim v$ и $v \sim w$. Тогда - существуют два рёберно непересекающихся пути

P_1, P_2 из u в v ($P_1 \cap P_2 = \emptyset$) - существуют два рёберно непересекающихся пути Q_1, Q_2 из v в w ($Q_1 \cap Q_2 = \emptyset$). Пусть $C = P_1 \cup P_2$. Заметим, что C — рёберно-простой цикл. Пусть вершины a и b — первые со стороны w вершины на пересечении $(Q_1 \cup Q_2) \cap C$. Рассмотрим пути $w \rightarrow a \rightarrow u$ и $w \rightarrow b \rightarrow u$, такие, что $a \rightarrow u$ и $b \rightarrow u$ идут по разные стороны по циклу C , а пути $w \rightarrow a$ и $w \rightarrow b$ идут по Q_1 и Q_2 соответственно. Приведённые пути не пересекаются, значит $u \sim w$. $\square$

Определение 3.2. Компонентой рёберной двусвязности в неориентированном графе называют класс эквивалентности по отношению рёберной двусвязности.

Определение 3.3. Мост в неориентированном графе — это ребро, при удалении которого увеличивается число компонент связности.

Теорема 3.2 (б/д). Рёбра графа конденсации по отношению рёберной двусвязности находятся во взаимно однозначном соответствии с мостами исходного графа: каждое ребро графа конденсации соответствует мосту исходного графа, и каждый мост исходного графа соединяет разные компоненты рёберной двусвязности, то есть является ребром графа конденсации.

Доказательство. Заметим сначала, что граф конденсации по отношению рёберной двусвязности ацикличесен. Действительно, если бы существовал цикл, то он бы образовывал новую компоненту рёберной двусвязности. Значит граф конденсации относительно рёберной двусвязности — лес. Заметим, что каждое ребро в дереве является мостом, поэтому все рёбра в графе конденсации будут мостами. Покажем теперь, что других мостов не существует. Действительно, пусть в компоненте рёберной двусвязности C есть ребро e , которое является мостом. Пусть e соединяет вершины x и y . Тогда x и y не рёберно двусвязны, поскольку граф конденсации ацикличесен, а также между долями C , разделёнными мостом e , все пути проходят через e . Следовательно, в компонентах рёберной двусвязности не может быть мостов. Таким образом, все рёбра в графе конденсации по отношению рёберной двусвязности являются мостами в исходном графе, и все мосты в исходном графе являются рёбрами в графе конденсации, а значит, находятся во взаимно однозначном соответствии. $\square$

Теорема 3.3 (критерий моста). В неориентированном графе древесное ребро $(p(v), v)$, где $p(v)$ — родитель v в дереве обхода DFS, является мостом, тогда и только тогда, когда $t_{up}(v) = t_{in}(v)$.

Доказательство. Обозначим ребро $e := (p(v), v)$.

$\Rightarrow$ Пусть ребро e — мост. Докажем, что $t_{up}(v) = t_{in}(v)$. Поскольку e — мост, его удаление нарушает связность графа: из любой вершины поддерева v невозможно достичь вершин вне этого поддерева без прохождения через ребро $(p(v), v)$. Следовательно, в поддереве v нет обратных рёбер выше $p(v)$. Следовательно, из определения $t_{up}(v)$ второй случай никогда не выполняется: нет вершин u выше $p(v)$, которые достижимы по обратному ребру из поддерева v . Значит, $t_{up}(v) = t_{in}(v)$.

$\Leftarrow$ Пусть $t_{up}(v) = t_{in}(v)$. Докажем, что ребро e — мост. Предположим обратное: ребро e не является мостом. Тогда существует путь P из v в $p(v)$, не использующий ребро e . Рассмотрим первую вершину x на пути P , которая не лежит в поддереве v (такая найдётся, так как $p(v)$ вне поддерева v). Пусть y — предшествующая ей вершина на пути P , тогда y лежит в поддереве v , а ребро (y, x) — обратное. Поскольку в обходе DFS вершина x была посещена раньше v , то $t_{in}(x) < t_{in}(v)$. Но тогда поскольку (y, x) — обратное ребро, то $t_{up}(v) \leq t_{in}(x) < t_{in}(v)$. Противоречие, так как по условию $t_{up}(v) = t_{in}(v)$. Следовательно, предположение неверно, и ребро e является мостом. $\square$

4 Точки сочленения

Определение 4.1. Две вершины в неориентированном графе вершинно двусвязны, если между ними есть два вершинно непересекающихся пути.

Замечание 4.1. Отношение вершинной двусвязности на вершинах не является отношением эквивалентности.

Доказательство. Покажем, что транзитивность не выполняется. Пусть граф задан на вершинах $\{u, v, w\}$ и имеет следующие рёбра: $(u, v), (u, v), (v, w), (v, w)$. Вершины u и v , а также v и w вершинно двусвязны, но вершины u и w не вершинно двусвязны. $\square$

Определение 4.2. Точка сочленения в неориентированном графе — вершина, при удалении которой увеличивается число компонент связности.

Теорема 4.1 (критерий точки сочленения). В неориентированном графе вершина v является точкой сочленения тогда и только тогда, когда:

1. v — корень дерева обхода DFS, и у v не менее двух детей в этом дереве.
2. v — не корень дерева обхода, и существует ребёнок w вершины v такой, что $t_{up}(w) \geq t_{in}(v)$.

Доказательство. Случай 1: v — корень DFS.

$\Rightarrow$ Пусть v — точка сочленения и корень. После удаления v граф распадётся на ≥ 2 компоненты связности. Каждая компонента содержит хотя бы одну вершину, достижимую из v первым шагом обхода, то есть ребёнка v . Если бы у v был только один ребёнок, все вершины бы лежали в одном поддереве, и после удаления v связность бы не нарушилась — противоречие. Следовательно, у v не менее двух детей.

$\Leftarrow$ Пусть у корня не менее двух детей w_1, w_2 . Пусть v не точка сочленения. Тогда после удаления v существует путь P между поддеревьями w_1, w_2 , не проходящий через v . Но при обходе DFS из v сначала в w_1 мы бы достигли поддерева w_2 , пройдя по пути P и добавили бы его как часть поддерева w_1 , а не как отдельного ребёнка — противоречие. Значит, v — точка сочленения.

Случай 2: v — не корень DFS.

$\Rightarrow$ Пусть v — точка сочленения. После удаления v граф распадётся на компоненты. Рассмотрим компоненту C , которая не содержит родителя $p(v)$. Все вершины C достижимы из v через некоторого ребёнка w (по древесному ребру (v, w)). Покажем, что $t_{up}(w) \geq t_{in}(v)$. Предположим противное: $t_{up}(w) < t_{in}(v)$. Тогда существует путь из поддерева w (то есть из C) в вершину u с $t_{in}(u) < t_{in}(v)$, не использующий ребро $(v, p(v))$. Поскольку $t_{in}(u) < t_{in}(v)$, вершина u была посещена до v , значит $u \notin C \cup \{v\}$. Этот путь соединяет C с внешней частью графа без прохождения через v — противоречие с тем, что C отдельная компонента после удаления v . Следовательно, $t_{up}(w) \geq t_{in}(v)$.

$\Leftarrow$ Пусть существует ребёнок w такой, что $t_{up}(w) \geq t_{in}(v)$. Следовательно, из поддерева w нет обратных рёбер в вершину u с $t_{in}(u) < t_{in}(v)$. Покажем, что после удаления v поддерево w станет новой компонентой связности. Предположим противное: существует путь P из вершины x в поддереве w в вершину $y \notin subtree(w) \cup \{v\}$, не проходящий через v . Рассмотрим первое ребро (a, b) на P , где $a \in subtree(w), b \notin subtree(w) \cup \{v\}$. Тогда $b \neq v$ и $t_{in}(b) < t_{in}(v)$ (иначе b вошла бы в поддерево w при DFS). Но тогда $t_{up}(w) \leq t_{in}(b) < t_{in}(v)$ — противоречие. Следовательно, поддерево w образует отдельную компоненту после удаления v , то есть v — точка сочленения. $\square$

5 Взвешенные графы

Определение 5.1. Взвешенным графом будем называть тройку $G = (V, E, w)$, где V — множество вершин ($|V| < \infty$), $E \subseteq V \times V$ — мультимножество рёбер, $w : E \rightarrow K \subseteq \mathbb{R}$ — весовая функция.

Определение 5.2. Весом (длиной) пути $p = v_1, \dots, v_k$ будем называть величину

$$w(p) = \sum_{i=1}^{k-1} w(v_i, v_{i+1})$$

Определение 5.3. Кратчайшим путём от вершины s до вершины t , $dist(s, t)$ будем называть путь $s = v_1, \dots, v_k = t$ такой, что его вес минимален среди всех возможных путей от s до t .

6 BFS

Определение 6.1. Обход в ширину (BFS, breadth-first search) — алгоритм обхода графа в порядке неубывания расстояния от заданной стартовой вершины s . В случае неориентированного графа без весов (либо с единичными весами рёбер) обход в ширину находит длины кратчайших путей от s до всех достижимых из s вершин.

Алгоритм 6.1 (BFS). Пусть даны граф $G = (V, E, w)$, $w : E \rightarrow K = \{1\}$ и стартовая вершина $s \in V$. Требуется заполнить массив расстояний $dist[v]$ — длина кратчайшего пути от s до v (или ∞ , если v недостижима).

Инициализация:

- Для всех $v \in V$ положить $dist[v] = \infty$, $used[v] = false$.
- $dist[s] = 0$, $used[s] = true$
- Создать очередь Q и поместить в неё s .

Основной цикл. Пока очередь Q не пуста:

- Извлечь вершину v из начала очереди.
- Для каждого соседа u вершины v если $used[u] = false$, то изменить $used[u] = true$, $dist[u] = dist[v] + 1$ и добавить u в конец очереди Q .

Псевдокод:

```

1 void BFS(Graph graph, Vertex start) {
2     Queue<Vertex> bfs_queue;
3     HashMap<Vertex, int> dist;
4     dist[start] = 0;
5     bfs_queue.push(start);
6     while (!bfs_queue.empty()) {
7         Vertex from = bfs_queue.pop();
8         for (Vertex to : graph.GetNeighbours(from)) {
9             if (dist.ContainsKey(to)) {
10                continue;
11            }
12            dist[to] = dist[from] + 1;
13            bfs_queue.push(to);
14        }
15    }
16 }
```

Теорема 6.1 (корректность BFS). В любой момент работы алгоритма BFS в очереди содержатся вершины, расстояния которых от стартовой вершины s равны только l или $l + 1$, причём все вершины с расстоянием l расположены перед вершинами с расстоянием $l + 1$, где l — расстояние от s до последней обработанной вершины (или $l = 0$, если очередь содержит только s).

Доказательство. Докажем утверждение по индукции по количеству обработанных вершин (т.е. вершин, извлечённых из очереди и помеченных как посещённые).

База. Изначально очередь содержит только вершину s , для которой $dist(s, s) = 0$. Таким образом, $l = 0$, и в очереди содержатся вершины с расстоянием $l = 0$. Утверждение выполнено.

Шаг. Пусть после обработки k вершин в очереди содержатся все вершины с расстоянием l (если они есть), все вершины с расстоянием $l + 1$ (если они есть), причём вершины с расстоянием l расположены

перед вершинами с расстоянием $l + 1$. Рассмотрим обработку $(k + 1)$ -й вершины v . По предположению индукции, v имеет расстояние l (поскольку вершины с расстоянием l расположены в начале очереди). При обработке v мы добавляем в очередь все ещё не посещённые соседние вершины u , для которых $\text{dist}(s, u) = \text{dist}(s, v) + 1 = l + 1$. Так как добавление происходит в конец очереди, новые вершины с расстоянием $l + 1$ размещаются после уже существующих вершин с расстоянием $l + 1$. Таким образом, упорядоченность очереди сохраняется: сначала вершины с расстоянием l , затем с расстоянием $l + 1$. $\square$

Теорема 6.2 (асимптотика BFS). *Алгоритм BFS работает за $O(|V| + |E|)$.*

Доказательство. Оценим, сколько раз в ходе алгоритма происходит перебор соседей вершины v . Каждое ребро $\{u, v\}$ хранится в списках смежности обеих вершин. Оно просматривается дважды: при обработке v и при обработке u . Общее количество просмотров $O(|E|)$. Также каждая вершина будет добавлена в очередь (и извлечена для перебора соседей) не более одного раза. Таким образом, операций с очередью $O(|V|)$ и суммарно $O(|V|) + O(|E|) = O(|V| + |E|)$. $\square$

7 0-1 BFS

Определение 7.1. 0-1 BFS — это модификация обхода в ширину для взвешенных графов, в которых вес каждого ребра принадлежит множеству $\{0, 1\}$. Алгоритм находит длины кратчайших путей от заданной стартовой вершины s до всех остальных.

Алгоритм 7.1 (0-1 BFS). В отличие от обычного BFS (который использует очередь), 0-1 BFS применяет дек:

- При переходе по ребру веса 0 вершина добавляется в начало дека — её расстояние не увеличивается, поэтому она должна быть обработана раньше текущих вершин того же уровня.
- При переходе по ребру веса 1 вершина добавляется в конец дека — её расстояние увеличивается на 1, поэтому она обрабатывается позже.

Псевдокод:

```

1 Vertex from = bfs_queue.pop_front();
2 for (Vertex to : graph.GetNeighbours(from)) {
3     if (dist.HasKey(to)) {
4         continue;
5     }
6     dist[to] = dist[from] + w(from, to);
7     if (w(from, to) == 0) {
8         bfs_queue.push_front(to);
9     } else {
10        bfs_queue.push_back(to);
11    }
12 }
```

Теорема 7.1 (корректность 0-1 BFS). *В любой момент времени работы алгоритма дек D содержит вершины, упорядоченные по неубыванию их текущих расстояний $\text{dist}[\cdot]$ от s .*

Доказательство. По индукции.

База. Изначально $D = [s]$, $\text{dist}(s, s) = 0$ — упорядоченность выполнена.

Шаг. Предположим, что перед обработкой вершины v дек упорядочен: все вершины в нём имеют расстояния $l, l, \dots, l, l + 1, l + 1, \dots, l + 1$. При обработке v с $\text{dist}(s, v) = l$:

- Для ребра веса 0 получаем вершину w с $\text{dist}(s, w) = l$. Она добавляется в начало дека, где уже находятся вершины с расстоянием l — порядок сохраняется.
- Для ребра веса 1 получаем вершину w с $\text{dist}(s, w) = l + 1$. Она добавляется в конец дека, где находятся вершины с расстоянием $> l$ — порядок сохраняется.

После извлечения v из начала дека упорядоченность также сохраняется. $\square$

Теорема 7.2 (асимптотика 0-1 BFS). *Алгоритм 0-1 BFS работает за $O(|V| + |E|)$.*

Доказательство. Анализ временной сложности аналогичен обычному BFS. Заметим, что каждая вершина может быть добавлена в дек несколько раз, но не более чем дважды:

- Первый раз — при получении расстояния $l + 1$ через ребро веса 1.
- Второй раз — при улучшении расстояния до l через ребро веса 0.
- Больше двух раз вершина добавляться не может, так как расстояния только уменьшаются, а минимальное расстояние фиксировано.

Таким образом, учитывая, что каждое ребро просматривается не более двух раз, и каждая вершина добавляется в дек не более чем дважды, время работы равно $O(|V| + |E|)$. $\square$

8 1-k BFS

Определение 8.1. 1-k BFS — это модификация обхода в ширину для взвешенных графов, в которых вес каждого ребра принадлежит множеству $\{1, 2, \dots, k\}$, где k — фиксированная константа. Алгоритм находит длины кратчайших путей от заданной стартовой вершины s до всех остальных вершин.

Алгоритм 8.1 (1-k BFS). В отличие от обычного BFS (который использует одну очередь), 1-k BFS применяет массив очередей `at_dist`, где `at_dist[d]` содержит очередь вершин с текущим расстоянием d . Это позволяет обрабатывать вершины в порядке неубывания расстояний, гарантируя корректность вычисления кратчайших путей.

Псевдокод:

```

1 int max_dist = k * (graph.GetVerticesCount() - 1);
2 Array<Queue<Vertex>> at_dist(max_dist);
3 at_dist[0].push(start);
4 HashMap<Vertex, int> dist;
5 dist[start] = 0;
6 for (int dist = 0; dist < max_dist; ++dist) {
7     while (!at_dist[dist].empty()) {
8         Vertex from = at_dist[dist].pop();
9         if (dist[from] < dist) {
10             continue;
11         }
12         for (Vertex to : graph.GetNeighbours(from)) {
13             if (dist[to] > dist[from] + w(from, to)) {
14                 dist[to] = dist[from] + w(from, to);
15                 at_dist[dist[to]].push(to);
16             }
17         }
18     }
19 }
    
```


Теорема 8.1 (корректность 1-k BFS). В любой момент работы алгоритма вершины в массиве очередей at_dist упорядочены по неубыванию их расстояний от s .

Доказательство. По индукции.

База. Изначально $at_dist[0] = [s]$, $dist(s, s) = 0$ — упорядоченность выполнена.

Шаг. Предположим, что перед обработкой расстояния l все вершины с расстоянием $< l$ уже обработаны, а $at_dist[l]$ содержит вершины с расстоянием l . При обработке вершины v с $dist(s, v) = l$:

- Для ребра веса $w \in \{1, \dots, k\}$ получаем вершину u с $dist(s, u) = l + w$.
- Вершина u добавляется в очередь $at_dist[l+w]$, которая будет обработана позже (так как $l+w > l$).
- Если $dist(s, u)$ уже меньше $l + w$, вершина пропускается.

Следовательно, вершины всегда обрабатываются в порядке неубывания расстояний, что гарантирует корректность значений $dist(s, v)$ как длин кратчайших путей. $\square$

Теорема 8.2 (асимптотика 1-k BFS). Алгоритм 1-k BFS работает за $O(k \cdot |V| + |E|)$.

Доказательство. Заметим, что каждое ребро просматривается один раз. Для ребра веса w вершина u может быть добавлена в очередь не более одного раза (после первого улучшения расстояния). Поскольку веса ограничены k , максимальное расстояние не превосходит $k \cdot (|V| - 1)$, что позволяет использовать массив фиксированного размера. Таким образом, асимптотика $O(k \cdot |V| + |E|)$. $\square$

Замечание 8.1. В каждый момент только k очередей не пусты (при обработке расстояния l нас интересуют только расстояния в диапазоне $[l, l+k]$), можно хранить k очередей вместо $k \cdot |V|$, и тогда потребление памяти составит $O(k + |V|)$.

9 0-k BFS

Определение 9.1. 0-k BFS — это модификация обхода в ширину для взвешенных графов, в которых вес каждого ребра принадлежит множеству $\{0, 1, \dots, k\}$, где k — фиксированная константа. Алгоритм находит длины кратчайших путей от заданной стартовой вершины s до всех остальных вершин.

Алгоритм 9.1 (0-k BFS). В отличие от 1-k BFS, 0-k BFS использует массив деков вершин at_dist , где $at_dist[l]$ содержит вершины с текущим расстоянием l от s . Отличие от 1-k BFS — обработка рёбер веса 0:

- При переходе по ребру веса 0 вершина добавляется в начало текущего дека $at_dist[l]$, чтобы быть обработанной раньше текущих.
- При переходе по ребру веса $w > 0$ вершина добавляется в конец дека $at_dist[l + w]$.

Теорема 9.1 (корректность 0-k BFS). При обработке дека $at_dist[l]$ все вершины с расстоянием $< l$ уже обработаны окончательно, а вершины в $at_dist[l]$ обрабатываются в порядке неубывания их окончательных расстояний.

Доказательство. Аналогично доказательству 1-k BFS и 0-1 BFS. $\square$

Теорема 9.2 (асимптотика 0-k BFS). Алгоритм 0-k BFS работает за $O(k \cdot |V| + |E|)$.

Доказательство. Аналогично доказательству 1-k BFS и 0-1 BFS. $\square$